Tutorial: How to Use Our Dynamic Fault Analysis Tool (FaultFlipper) on C Programs and Binaries

Ryan Evans, Zach Leech, and Boyang Wang

Data and System Security Lab (DaSec), ECE Department, University of Cincinnati
Last modified: June 2026, Status: Completed

1 Introduction

Overview. This document explains how to perform dynamic fault analysis on binary files written in the C programming language using our in-house tool named *FaultFlipper*. Dynamic fault analysis against simulation target architectures can uncover vulnerabilities in a binary program. Fault analysis over binaries can help designers identify lines in C code that are susceptible to event upsets and mitigate those vulnerabilities at the development stage. It is worth mentioning this is a *white-box analysis*, which means we need to have access to the C source code. This tutorial provides basic instructions on how to use our tool with two simple example C programs: Password Check and Fibbonaci.

Source Code: Our source code can be found at https://github.com/UCdasec/FaultFlipper

Reference: When reporting results that use our code in the above repository, please cite our research paper below:

Ryan Evans, Brendan Kickpatrick, Minh Khiem Ha, Prateek Kharangate, Boyang Wang, "FaultFlipper: A Dynamic Fault Analysis Tool," the 7th workshop on Artificial Intelligence in Hardware Security (AIHWS 2026) in conjunction with ACNS 2026, June 22-25, Stony Brook, New York, USA.

Acknowledgments. The research and education efforts associated with this document are supported by CHEST – NSF IUCRC Center in Hardware and Embedded Systems Security and Trust.

2 Our Dynamic Fault Analysis Tool

Overview of Our Dynamic Fault Analysis. In order to perform dynamic fault analysis, our tool, FaultFlipper, leverages the following workflow: given a C source file and an program input as the inputs of our tool, our tool first compiles the binary based on a specified architecture (x86, ARM, or RISC-V). This binary is known as the *golden binary*, which is then executed with the QEMU emulator to obtain information related to this program's output and executation. This execution is also known as the golden run. The execution of this binary produces three primary artifacts: the program's `STDOUT`, `return_code`, and a sequence of instruction addresses that were executed.

Next, our tool leverages the `lief` library to parse the instructions from the golden binary, and then generates mutants, where each mutant is a mutated binary on one instruction or one bit

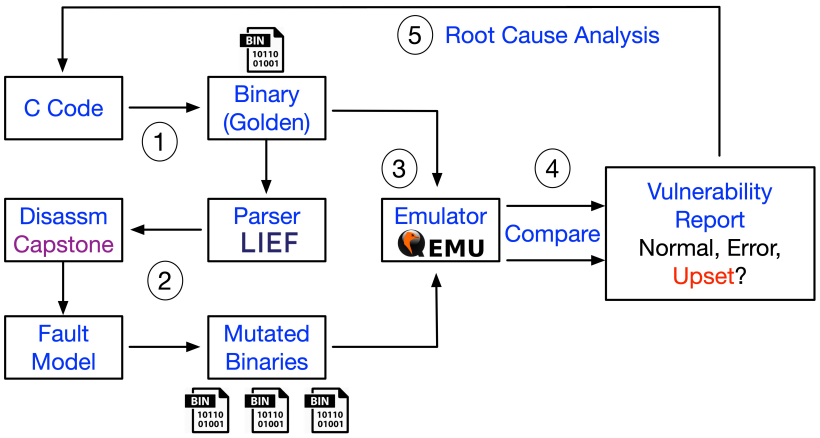


Fig. 1: Overview of our dynamic fault analysis pipeline

from the golden binary. The generation of mutants is determined by a given *fault model* that our tool supports. Once all the mutants are produced based on a fault model, our tool executes each mutant with QEMU, records its three artifacts, including `STDOUT`, `return_code`, and a sequence of instruction addresses that were executed. Our tool compares the artifacts of each mutant with the ones from the golden binary to decide whether a mutant belongs to one of the three cases:

1. **Normal**: The execution of the mutated binary completes, the `STDOUT` and `return_code` are the same as the ones from the golden binary.
2. **Error**: The execution of the mutated binary resulted in an error, where there was a segmentation fault or its `return_code` is different from the one obtained from the golden binary.
3. **Upset**: The execution of the mutated binary completes; the `return_code` is the same as the golden binary but its `STDOUT` is different from the golden binary.

It is worth mentioning that the decision of the type of a mutant is *program-dependent* and *input-dependent*, which users can customize within our tool.

Fault Models for Mutation Generation. FaultFlipper currently supports two major fault models, including instruction skip (NOP) and bit flips (BIT).

Instruction Skipping (NOP). This fault model simulates the case where the execution of an instruction is skipped due to glitching. To simulate this effect, a typical approach is to replace one original instruction in the golden binary with NOP instruction(s), where the number of bytes of the NOP instructions is the same as the number of bytes of the original instruction.

The remaining instructions remain the same in the binary. This allows an emulator to skip one instruction and continue to execute the remaining instructions without additional changes to the binary.

The NOP instruction is architecture dependent. For instance, for ARM, `0x00 0x00 0xA0 0xE1` can be used as a NOP instruction. In RISC-V, `0x12 0x00 0x00 0x00` is used as a NOP instruction.

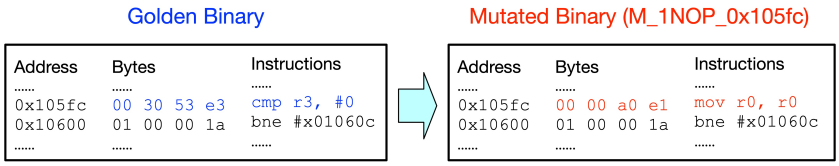


Fig. 2: One mutated binary produced with 1-NOP fault model.

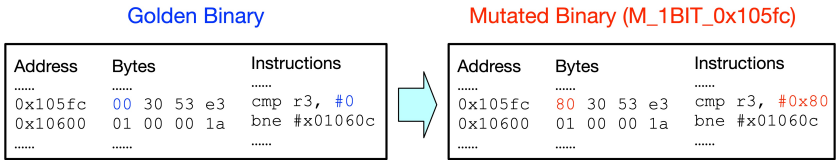


Fig. 3: One mutated binary produced with 1-BIT fault model.

We denote this fault model as **1-NOP** if we only replace one instruction with NOP(s) in a binary. If there are N instructions from the `.text` section, then N mutated binaries will be produced from this 1-NOP fault model, where each mutant is one-instruction-different compared to the golden binary. An example is presented in Fig. 2. We can also extend this fault model to replace an X number of *consecutive instructions* in a binary with NOPs. We denote the fault model as **X-NOP**.

Bit Flips (BIT). This fault model simulates the case where a single bit of a binary are flipped due to faults of hardware, software, or memory (e.g., fault injection, Rowhammer, etc.). To simulate this fault, we flip one bit in the binary when generating a mutant and we enumerate every single bit to generate all the mutants. An example is illustrated in Fig. 3.

We denote it as **1-BIT**. If there are N instructions from the *.text* section of a binary, then a number of $32 \times N$ mutated binaries will be initially produced (if each instruction consists of 32 bits). Since not every mutant produced from this fault model will be executable, we validate the mutated instruction in each binary by decoding it with `Capstone` library and filter out the ones with invalid instructions. We only pass the mutants with valid instructions to the emulator. In addition, we also extend this fault model to flip an X number of *consecutive bits* in a binary. We denoted it as **X-BIT**.

Source Code and Example Datasets. In addition to the description mentioned in this document, we also provide source code, tutorial videos, and links to example datasets on our GitHub repository (`https://github.com/UCdasec/FaultFlipper`) for reproducibility. We recommend that readers read this tutorial document first, and then review the videos.

3 Software and Hardware Settings

We use the following hardware and software in our simulation pipeline. We use a Linux machine for the instructions in this document. However, the tool is also compatible with Windows.

- Hardware: A desktop with an Intel i9 CPU, 128 GB memory, and an NVIDIA RTX 4080 GPU running Linux (Ubuntu 22.04)
- Software (all free or open-source): Python 3, and some other Python libraries

3.1 Install FaultFlipper Project

The description to install the required software is below. Specifically, FaultFlipper itself needs downloading and package installation, as well as the `QEMU` emulator.

FaultFlipper is a package as a typical Python package, leveraging the `pixi` package manager for dependency management (however, you may use any related package manager of your choice).

```
# To install pixi if you do not have it
curl -fsSL https://pixi.sh/install.sh | sh

# Clone FaultFlipper
git clone git@github.com:UCdasec/FaultFlipper.git
cd FaultFlipper

# Download Deps and enter  shell
pixi shell
```

FaultFlipper requires the `QEMU` emulator to be installed as well. The following package will install the required emulator components for all supported targets: x86, ARM, and RISC-V, each with 64bit and 32bit support.

```
sudo apt update
sudo apt install qemu-user
```

4 Run Our Tool with an Example - Password Check

In this section, we describe the steps to configure and run our dynamic fault analysis FaultFlipper on an example C source file. Specifically, we will use a Password Checker source file.

Intro to Password Checker Example. The password checker file is a simple password checking program. It is written in C source code, and carries decision making logic that is potentially vulnerable to fault injection attacks. When executes, the program asks the user to provide an input password. The user input is then compared to the correct password (hard-coded), and if correct, the program prints `Correct`, otherwise the program prints `Wrong`. The exact program is shown below:

```
#include <stdio.h>
#include <stdlib.h> // Required for the exit() function
#include <stdbool.h>
#include <string.h>
#include <stdbool.h>
#define PASSWORD 'pass' // The hard-coded correct Password
#define MAX_LENGTH 10 //password max length

bool password_check(char input[]) {
    bool flag = 0;
```

```
    if (strncmp(input, PASSWORD, MAX_LENGTH) == 0) {
        flag = 1;
    }
    // Important decision
    if (flag == 1) {
        printf('Correct\n');
    } else {
        printf('Wrong\n');
    }
    return flag;
}

int main(){
    // Buffer to store user input (extra byte for null terminator)
    char input[MAX_LENGTH + 1];
    printf('Enter the password (max %d characters): ', MAX_LENGTH);
    bool flag = 0;
    // fgets takes arguments: (buffer, buffer_size, input)
    // this assures that the input is no longer than the size of the buffer
    if (fgets(input, sizeof(input), stdin) != NULL) {
        // If the entered password exceeds the
        // buffer it's incorrect
        if (strchr(input, '\n') == NULL) {
            flag = 0;
        } else { // Remove the newling character
            input[strcspn(input, '\n')] = '\0';
        };
        flag = password_check(input);
    };
    return 0;
}
```

4.1 Understand FaultFlipper Usage

FaultFlipper is implemented as a command-line-interface (CLI) tool. That is, it operates in the terminal, and provides commands to perform that fault simulations. To run the program:

```
>>> pixi run cli
```

This will output something similar to Figure 4 (subject to change, not all commands will be the exact same). This will list the set of commands that FaultFlipper supports. In this example run, we will be specifically using the `x-nop` command, which simulates a fault campaign with our X-NOP fault model.

Commands take arguments, and to see what arguments a command needs, we can run:

```
❯ pixi run cli
✨ Pixi task (cli): python src/faultflipper/cli.py                                    Usage: cli.py COMMAND

┌─ Commands ──────────────────────────────────────────────────────────────────────────────────────────────┐
│ angr-nop-no-comp-inout    This version of the experiments takes tuples of: [ (INPUT, EXPECTED_OUTPUT), ....]│
│ bit-no-comp-inout         Run a bit experiment on a already compiled binary with (in,out) tups.         │
│ compare-regs              Temp function to get the registers of all functions                           │
│ filter-results            Filter an existing experiment's results.csv down to the provided addresses.   │
│ find-faulted              From the results file find the binaries that had the exptected STDOUT then print the│
│                           dissassembly comparison between all those programs and the base program       │
│ gather-reports            If there are many report.md files in a directory, this will 1. Gather them 2. Rename 3.│
│                           Save in the output directory                                                  │
│ generate-exp-file         Save a toml that defines the experiment for the neural networks               │
│ get-disasm                An over-engineered function to re-create objdump... but this allows me to easily inject│
│                           its results into a PDF report! :D (and also make the output colorful)         │
│ get-overhead              Compare the overhead of files                                                 │
│ nn-generate-exp-files     A temporary function to generate experiemnt files for classifier testing      │
│ nop-no-comp-inout         This version of the experiments takes tuples of: [ (INPUT, EXPECTED_OUTPUT), ....]│
│ read-results              Read the results.csv of and experiemnt                                        │
│ run                       This will run ALL the experiments in the provided experiment file             │
│ spectral-plot             Make the spectrral plot.                                                      │
│ x-bit                     Run the bit experiemnt with either the qemu backend or the angr backend. Set  │
│                           delete_non_upsets to remove mutated binaries that behaved normally or errored when the│
│                           run completes.                                                                │
│ x-bit-qemu-parallel-data  Run an experiment that gernerates mutant binaries overriding the target section│
│ x-nop                     Command to run NOP experiments. Set delete_non_upsets to remove mutated binaries that│
│                           did not produce an event upset after results are collected.                   │
│ --help -h                 Display this message and exit.                                                │
│ --version                 Display application version.                                                  │
└─────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Fig. 4: output from pixi run cli command

```
❯ pixi run cli x-nop --help
✨ Pixi task (cli): python src/faultflipper/cli.py x-nop --help
Usage: cli.py x-nop [ARGS] [OPTIONS]

Command to run NOP experiments. Set delete_non_upsets to remove mutated binaries that did not produce an event upset after results are collected.

┌─ Parameters ──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ *  PROGRAM-FILE --program-file                          [required]                                                                                          │
│ *  OUT-DIR --out-dir                                    [required]                                                                                          │
│ *  PROGRAM-INPUT --program-input                        [required]                                                                                          │
│ *  EXPECTED-STDOUT --expected-stdout                    [required]                                                                                          │
│ *  EXPECTED-RETURNCODE --expected-returncode            [required]                                                                                          │
│    LIST-EXPECTED --list-expected --no-list-expected     [default: False]                                                                                    │
│    TIMEOUT --timeout                                    [default: 5]                                                                                        │
│    SAVE-RESULTS --save-results                                                                                                                              │
│    YES --yes --no-yes                                   [default: False]                                                                                    │
│    PROGRAM-SOURCE-CODE --program-source-code                                                                                                                │
│    DYNAMIC-FILTER --dynamic-filter --no-dynamic-filter  [default: False]                                                                                    │
│    COMP --comp --no-comp                                [default: True]                                                                                     │
│    OPTS --opts                                                                                                                                              │
│    TRACE-BACKEND --trace-backend                        [choices: angr, best] [default: best]                                                               │
│ *  TARGET --target                                      [choices: x86-32, riscv, arm-64, arm-32, riscv-32, x86-64] [required]                               │
│    FUNC-NAMES --func-names                              [default: ]                                                                                         │
│    NUM-NOPS --num-nops                                  [default: 1]                                                                                        │
│    NUM-CPUS --num-cpus                                  [default: 1]                                                                                        │
│    VERBOSE --verbose --no-verbose                       [default: False]                                                                                    │
│    BACKEND --backend                                    [choices: angr, qemu] [default: qemu]                                                               │
│    LOG-MATCHING --log-matching --no-log-matching        [default: True]                                                                                     │
│    OPTIMIZATION --optimization                          [choices: o0, o1, o2, o3, oz] [default: o0]                                                         │
│    DELETE-NON-UPSETS --delete-non-upsets                [default: False]                                                                                    │
│      --no-delete-non-upsets                                                                                                                                 │
│    ADDRESSES-JSON --addresses-json                                                                                                                          │
└─────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Fig. 5: output for pixi run cli x-nop –help command

```
>>> pixi run cli x-nop --help
```

This will output a list of parameters, seen in Figure 5. The parameters at the top of the list with a red star to the left of the parameter name, indicating the parameter is required. For the `x-nop` command this includes: `program-file`, `out-dir`, `program-input`, `expected-stdout`, `expected-returncode`, and `target`. While we can run each fault command manually, we recommend to configure an experiment profile (described in the next section), which allows us to reuse and modify the command with different settings easily. Details regarding how to run it with a experiment profile are present next.

4.2 Configure Fault Analysis Parameters in Profiles (Optional)

FaultFlipper can also run with an **experiment profile**, e.g., a `.toml` file. This enables experiment settings to be saved and re-used. To run an experiment based on a profile *all command parameters* must be defined in a compliant `toml` file. For example, below is a list of all the fields needed in a experiment profile for running the `x-nop` command:

- `command`: (type: string) The FaultFlipper command to run.
- `program-file`: (type: string): The source code for the target program.
- `program-input`: (type: string) The input through STDIN to the program.
- `expected-returncode`: (type: int) The expected return code of the program.
- `expected-stdout`: (type: string) The expected STDOUT of the program. *Please note that, this can be the expected STDOUT of Normal behavior if we set `UPSET ON MATCH` as false; or it is the expected STDOUT of Upset behavior if we set `UPSET ON MATCH` as true.*
- `list-expected`: (type: bool) List the mutated programs that had the expected STDOUT once the simulation is complete.
- `timeout`: (type: int) Timeout time in seconds for running the mutated programs (to avoid the cases that a mutant runs into an error state without stopping).
- `out-dir`: (type: string) Location of simulation results.
- `yes`: (type: bool) If TRUE skip asking the user for permission to begin run after calculating number of mutations, if False prompt the user to ask for permission to run.
- `target`: (type: string) The compilation target
- `num_cpus`: (type: int) The number of CPU cores to use (e.g., 1 or 24)
- `num_nops`: (type: int) The number of consecutive NOPs to insert (e.g., 1 or 2)
- `upset_on_match`: (type: bool) if `true`, `expected-stdout` is the expected output of Upset; if `False`, `expected-stdout` is the expected output of Normal. *This command parameter is case sensitive, so the boolean input must be lowercase.*

In the following configuration, we apply the `x-nop` model to the password check program. Given an incorrect password (by assigning `program-input` as “test", an Event upset is a mutated binary

with the `expected-stdout` is “Correct" and `expected-returncode` is 0, if we set `upset_on_match` as True. The default optimization level is o0, which cannot be changed in the configuration file. If we would like to change it, we can use the ‘x-nop’ command via the `-optimization` flag.

```
[experiment.nop1_pass]
command = "x_nop"
program-file = "test_files/password_check.c"
program-input = "test\n"
expected-returncode = 0
expected-stdout = "Correct"
list-expected = false
timeout = 4
out-dir = "pass_exp_output"
yes = true
target = "arm_32"
num_cpus = 24
num_nops = 1
upset_on_match = true
```

The file is saved as `pass_profile.toml` and used in the next step to perform dynamic analysis. If we would like to apply for a different fault model or with a different parameter, we can update the parameters in `pass_profile.toml` file.

4.3 Run Our Dynamic Analysis Tool

The fault analysis is performed automatically by our tool. To run our tool, we can run the command line directly,

```
>>> pixi run cli x-nop --program-file test_files/password_check.c --out-dir pass_exp_output
↪  --program-input "test\n" --expected-stdout "Correct" --expected-returncode 0 --timeout 4
↪  --target arm_32 --num-cpus 24 --yes
```

In addition, we can also run it by passing the configuration file `pass_profile.toml` created in the last step to FaultFlipper with the below command. Either running the command line directly or using .toml file will produce the same results (if we use the same set of parameters and arguments).

```
>>> pixi run cli run pass_profile.toml
```

After starting the tool, the tool will produce an error if the configuration file is incorrect, otherwise, the program will begin. If the parameter `yes` in the `.toml` file was set to false, the program will first calculate the total number of faults and ask the user if the user would like to proceed (this can be useful for understanding the scale of the evaluation, especially when running large programs).

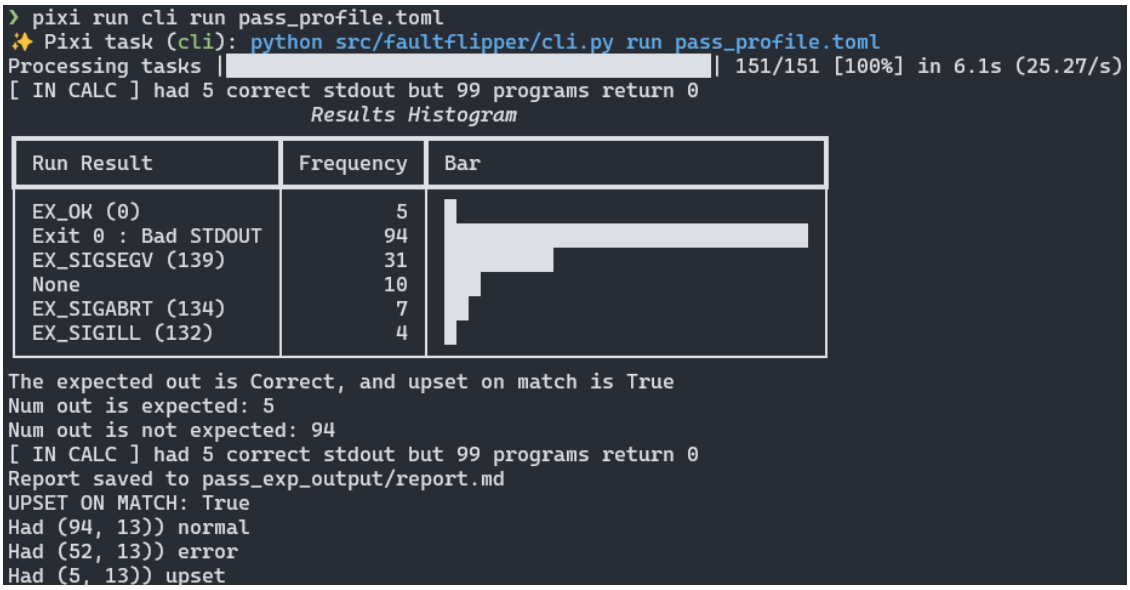


Fig. 6: Success run of our tool with Password Check program

Otherwise, the program will proceed directly to mutation generation and running mutated binaries. An example of a successful run is shown in Figure 6.

The results of the analysis show that 5 mutated binaries with `EX_OK (0)`, indicating these binaries complete with return code defined in `expected-returncode = 0` and the output of these mutated binaries are the same as defined in `expected-stdout = Correct`. Given `UPSET ON MATCH` is ON in this example, it means that each of them is considered as an Event Upset given an incorrect password. In addition, our tool also shows that **a total of 151 mutants were executed, 94 of them were normal, 52 of them were error, and 5 were upset** (i.e., an incorrect password can still bypass the verification due to the fault model with instruction skipping, mostly due to skipping branch instructions related to decision making). We can also observe that the dynamic analysis process took only 6.13 seconds to complete for this password check example.

4.4 Examine the Results of Dynamic Fault Analysis

At this point, the entire dynamic fault analysis process has been completed. We can then review the final report named, `OUTPUT_DIR/report.pdf`, where the `OUTPUT_DIR` is the parameter value defined by `out-dir` in the `toml` file. There are other files saved to the `out-dir`, including all the mutated binaries, the C source file, the golden binary, a markdown version of the report, a CSV file containing the analysis results, the experiment parameters saved in a json file, and a list of vulnerable C lines of the source file saved in a json file. A screenshot of the list of these files generated by the analysis of this password example is in Figure 7.

```
❯ ls pass_exp_output
experiment_parametes.json  password_check.c  report.md   results.csv
mutated_bins               password_check.o  report.pdf  vuln_c_lines.json
```

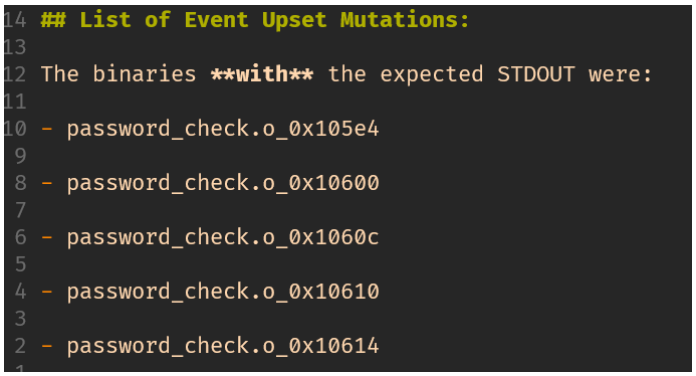
Fig. 7: Experiment artifacts (from Password Check Example)

The generated report records all experiment parameters, run time information, and experiment results. The numbers of Normal, Error and Upsets are also included. Specifically, the report contains the following sections:

1. **Settings**: Includes all experiment settings provided in the configuration file or passed to the cli command.
2. **Binary Information**: Reports the information about the golden binary, including the number of bytes, the number of bits, the total number of attempted mutants, the number of valid mutants, the target architecture, the command used to compile the golden binary, optimization level, the command used to run the binary, and the bytes used for the NOP instruction.
3. **Results at a Glance**: The number of Normal, Error, and Upset cases.
4. **Return Code Frequency:** Reports the different `reutrn_codes` and their frequency observed during the emulation.
5. **List of Event Upset Mutations**: A list of the names of the mutated binaries that resulted in Upset (a name of a mutated binary is in the format of [program_name.o_fault_address], e.g., `password_check.o_0x105e4`, indicating the fault happens on the instruction at address 0x10534).
6. **Root Cause Analysis**: A mapping of the assembly instructions associated with upsets to the lines in the C source file.
7. **List of Original Program vs Event Upset Mutated Programs**: Reports the disassemble lines in the golden binary and mutated binary around a given address, where the given address resulted in an event upset when mutated.
8. **Source Code Lines**: Reports the source code from the program file.

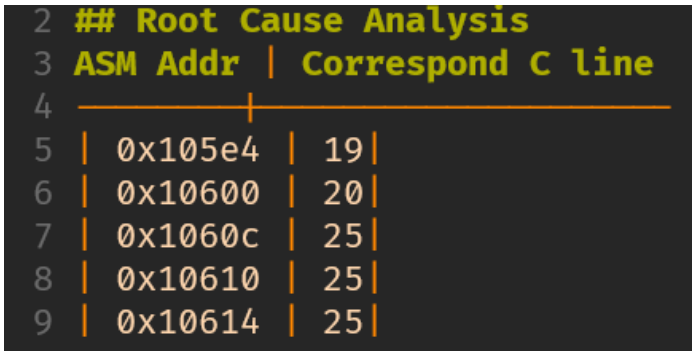
As mentioned, the main results are contained within the *report.md* file. Examples of the names of the mutated binaries leading to upsets are presented in Fig. 8. Examples of the addresses triggering upsets and their associated C lines in the source file are shown in Fig. 9. These 5 upsets map to three lines, line 19, 20, and 25 in the C source code, which are highlighted in Table 1. In addition, examples of detailed instructions associated with an upset are presented in Fig. 10.

Given the results, we can manually run mutated binaries to further validate upsets. For instance, we can run the mutated binary `password_check.o_0x105e4` by providing an incorrect password “wrong", which results in printing out “Correct".



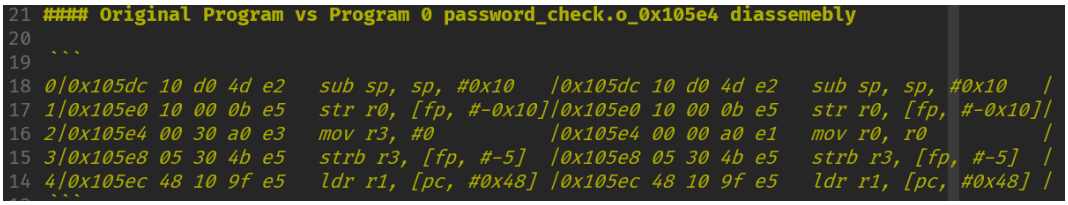
```
## List of Event Upset Mutations:

The binaries **with** the expected STDOUT were:

- password_check.o_0x105e4

- password_check.o_0x10600

- password_check.o_0x1060c

- password_check.o_0x10610

- password_check.o_0x10614
```

Fig. 8: Names of mutated binaries with upset reported in report.md



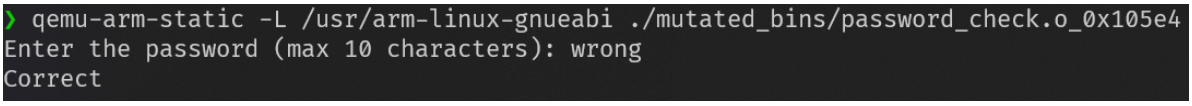
```
## Root Cause Analysis
ASM Addr | Correspond C line
---------|-----------------
| 0x105e4 | 19|
| 0x10600 | 20|
| 0x1060c | 25|
| 0x10610 | 25|
| 0x10614 | 25|
```

Fig. 9: Lines in the source file associated with update mutants in report.md



```
#### Original Program vs Program 0 password_check.o_0x105e4 diassemebly

 ```
0|0x105dc 10 d0 4d e2   sub sp, sp, #0x10 |0x105dc 10 d0 4d e2   sub sp, sp, #0x10 |
1|0x105e0 10 00 0b e5   str r0, [fp, #-0x10]|0x105e0 10 00 0b e5   str r0, [fp, #-0x10]|
2|0x105e4 00 30 a0 e3   mov r3, #0 |0x105e4 00 00 a0 e1   mov r0, r0 |
3|0x105e8 05 30 4b e5   strb r3, [fp, #-5] |0x105e8 05 30 4b e5   strb r3, [fp, #-5] |
4|0x105ec 48 10 9f e5   ldr r1, [pc, #0x48] |0x105ec 48 10 9f e5   ldr r1, [pc, #0x48] |
 ```
```

Fig. 10: Disassembly of mutated file `password_check.o_0x105e4`, where the instruction in the golden binary at address 0x10504 is `mov r3 #0` while the instruction from the mutated binary at the same address is `move r0 r0`



```
❯ qemu-arm-static -L /usr/arm-linux-gnueabi ./mutated_bins/password_check.o_0x105e4
Enter the password (max 10 characters): wrong
Correct
```

Fig. 11: Manually running a mutated program

Table 1: Map Vulnerable Assembly to the Original C Code (`password_check.c`, Arm32, fault model: 1-NOP)

Address	Original Inst.	Mutated Inst.	Line #	Original C Code
0x105e4	mov r3, #0	mov r0, r0	19	`bool flag = 0;`
0x10600	bne #0x1060c	mov r0, r0	20	`if (strncmp(input, PASSWORD, MAX_LENGTH) == 0)`
0x1060c	ldrb r3, [fp, #-5]	mov r0, r0	25	`if (flag == 1)`
0x10610	cmp r3, #0	mov r0, r0	25	`if (flag == 1)`
0x10614	beq #0x10624	mov r0, r0	25	`if (flag == 1)`

To reduce the runtime of the analysis, FaultFlipper supports a *tracing feature*. The tracing feature runs the golden binary once, and records a trace of all the executed instructions as well as the addresses of these instructions. Using this trace of executed instruction addresses, FaultFlipper then generates mutated binaries where only instructions that were executed will be mutated. This reduces the total number of mutated binaries that will need to be executed. To run with this tracing feature, the `x-nop` command is used, where the flag `-dynamic-filter` is passed[1]. For example, to run the equivalent fault campaign as above using the `x-nop` command but with the tracing feature, we can run the command below:

```
>>> pixi run cli x-nop --program-file test_files/password_check.c --out-dir pass_exp_output
↪  --program-input "test\n" --expected-stdout "Correct" --expected-returncode 0 --timeout 4
↪  --target arm_32 --num-cpus 24 --optimization o0 --dynamic-filter --yes --num-nops 1
```

4.5 Run with X-BIT Fault Model

We can also run this password check program with our bit flipping (X-BIT) fault model. The configuration file `.toml` file can be defined as below with the bit fault model. Specifically, compared to the configuration file used for X-NOP, we update the `command = ‘‘x-bit"`, remove `num_nops = 1`, add `num_bits = 1`, and save to a new output directory of `bit_pass_output`.

```
[experiment.bit1_pass]
command = "x_bit"
program-file = "test_files/password_check.c"
program-input = "test\n"
expected-returncode = 0
expected-stdout = "Correct"
list-expected = false
timeout = 4
out-dir = "bit_pass_output"
yes= true
target = "arm_32"
num_cpus = 24
num_bits = 1
upset_on_match = true
```

After updating the file, we can re-run the analysis and obtain a new set of results. Given the above profile is now saved to path `bit_pass_profile.toml` The command to run the above profile is:

```
>>> pixi run cli run bit_pass_profile.toml
```

The corresponding `x-bit` command for the above profile is:

[1] As of June 2026, we currently have an error with –dynamic-filter option due to the updated version of QEMU in Ubuntu, can skip it for now if there are errors. We are actively seeking solutions for it

```
>>> pixi run cli x-bit --program-file test_files/password_check.c --out-dir bit_pass_output
↪  --program-input "test\n" --expected-stdout "Correct" --expected-returncode 0 --timeout 4
↪  --target arm_32 --num-cpus 24 --optimization o0  --dynamic-filter --yes --num-bits 1
```

5 Run Our Tool with an Example - Fibonacci

In this section, we show how to expand our tool to run another example program, named Fibonnaci program, which calculates and outputs a number in the Fibonnaci sequence given an index as input. For the ease of demonstration, we set hard-coded the index as 20 (but one can always change it). This is considered as a vulnerable program as a fault may affect the number of iterations executed, which may lead to an incorrect output (i.e., an upset in this case). The source code of the program is shown below:

```c
#include <stdio.h>
unsigned long long fib(int a ) {
    // The number of iterations in fibonnaci to run
    int n = a;

    // Handle simple edge cases upfront:
    if ((n == 0) || (n<0)) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }

    unsigned long long prev = 0;
    unsigned long long curr = 1;
    unsigned long long next;

    // Run the loop
    for (int i = 2; i <= n; i++) {
        next = prev + curr;
        prev = curr;
        curr = next;
    }

    // Print the results 6765
    printf('Fibonacci of %d is: %llu\n', n, curr);

    return curr;
}

int main(){
    int a = 20;
```

```
    unsigned long long b = fib(a);

    return 0;
}
```

5.1 Run with X-NOP Fault Model

Similar as the password check example, we can create a configuration file, `fib_profile.toml`, to analyze the Fibonnaci example. Specifically, we use the `x-nop` fault model by setting `command = 'x_nop'`, where x is 1 (i.e., `num_nops = 1`). The input to the program is hard-coded as `int a = 20` already so we can leave the program-input as empty. The STDOUT is set as 6765 assuming the program behaves normally as expected. `Upset On Match` is set as False. In other words, if the output of a mutated binary is not 6765, it is considered as an upset.

```
[experiment.nop1_fib]
command = 'x_nop'
program-file = 'test_files/fib.c'
program-input = ''
expected-returncode = 0
expected-stdout = '6765'
list-expected = false
timeout = 4
out-dir = 'fibonnaci_output'
yes= true
target = 'arm_32'
num_cpus= 24
num_nops= 1
upset_on_match = false
```

To run the fault emulation, we can pass the configuration file to our tool FaultFlipper as below:

```
>>> pixi run cli run fib_profile.toml
```

The equivalent `x-nop` command for the above profile is:

```
>>> pixi run cli x-nop --program-file test_files/fib.c --program-input '' --expected-returncode
↪  0 --expected-stdout 6765  --no-list-expected --timeout 4 --out-dir fibonnaci_output --yes
↪  --target arm-32 --num-cpus 24 --num-nops 1 --no-upset-on-match
```

If the configuration file is defined properly, and our tool runs, we can observe similar as in Fig. 12. It shows that the analysis runs for 12.1 seconds. A total of 142 mutated binaries were executed, in which 65 of them are normal, 43 of them are error, and 34 of them are upset.

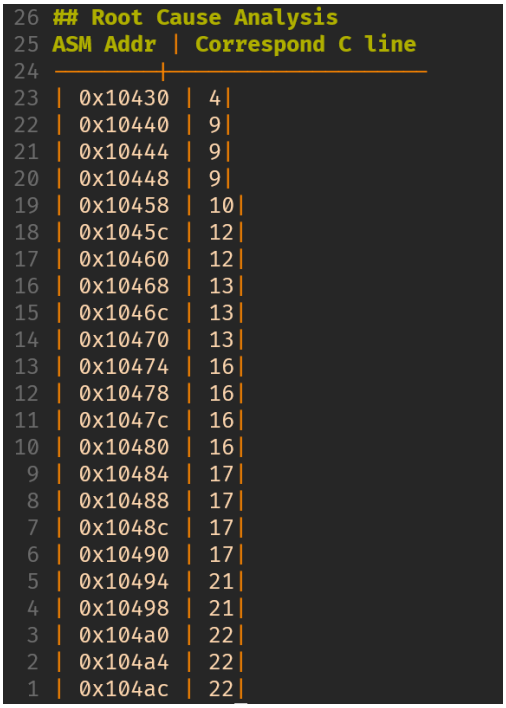
```
❯ pixi run cli run fib_profile.toml
✨ Pixi task (cli): python src/faultflipper/cli.py run fib_profile.toml
Processing tasks |                                        | 142/142 [100%] in 12.1s (11.83/s)
[ IN CALC ] had 65 correct stdout but 99 programs return 0
                        Results Histogram
┏━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━┳━━━━━┓
┃ Run Result         ┃ Frequency ┃ Bar ┃
┡━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━╇━━━━━┩
│ EX_OK (0)          │        65 │     │
│ Exit 0 : Bad STDOUT│        34 │     │
│ None               │        23 │     │
│ EX_SIGSEGV (139)   │        15 │     │
│ EX_SIGILL (132)    │         2 │     │
│ 109                │         2 │     │
│ 188                │         1 │     │
└────────────────────┴───────────┴─────┘
The expected out is 6765, and upset on match is False
Num out is expected: 65
Num out is not expected: 34
[ IN CALC ] had 65 correct stdout but 99 programs return 0
Report saved to fibonnaci_output/report.md
UPSET ON MATCH: False
Had (65, 13)) normal
Had (43, 13)) error
Had (34, 13)) upset
```

Fig. 12: Success run of our tool with Fibonacci program (1-NOP)

```
❯ ls fibonnaci_output
experiment_parametes.json  fib.o         report.md   results.csv
fib.c                      mutated_bins  report.pdf  vuln_c_lines.json
```

Fig. 13: Experiment artifacts (from Fibonacci Example)

We can observe similar information in the *report.md*. Examples of the addresses triggering upsets and their associated C lines in the source file are shown in Fig. 14.



```
## Root Cause Analysis
ASM Addr | Correspond C line
---------|-------------------
| 0x10430 | 4|
| 0x10440 | 9|
| 0x10444 | 9|
| 0x10448 | 9|
| 0x10458 | 10|
| 0x1045c | 12|
| 0x10460 | 12|
| 0x10468 | 13|
| 0x1046c | 13|
| 0x10470 | 13|
| 0x10474 | 16|
| 0x10478 | 16|
| 0x1047c | 16|
| 0x10480 | 16|
| 0x10484 | 17|
| 0x10488 | 17|
| 0x1048c | 17|
| 0x10490 | 17|
| 0x10494 | 21|
| 0x10498 | 21|
| 0x104a0 | 22|
| 0x104a4 | 22|
| 0x104ac | 22|
```

Fig. 14: Lines in the source file associated with update mutants in report.md

5.2 Run with X-BIT Fault Model

We can also run this Fibonnaci program with our bit flipping (X-BIT) fault model. The configuration file `.toml` file can be defined as below with the bit fault model. Specifically, compared to the configuration file used for X-NOP, we update the `command = “x-bit"`, remove `num_nops = 1`, and add `num_bits = 1`.

```
[experiment.bit1_fib]
command = "x_bit"
program-file = "test_files/fib.c"
program-input = ""
expected-returncode = 0
expected-stdout = "6765"
list-expected = false
timeout = 4
out-dir = "fibonnaci_output"
yes= true
```

Table 2: Map Vulnerable Assembly to the Original C Code (`fibonaaci.c`, Arm32, fault model: 1-NOP)

Address	Original Inst.	Mutated Inst.	Line #	Original C Code
0x1043c	cmp r3, 0	mov r0, r0	9	`if ((n == 0) || (n<0)) {`
0x1044c	bge 0x1045c	mov r0, r0	9	`if ((n == 0) || (n<0)) {`
0x10464	bne 0x10474	mov r0, r0	12	`if (n==1) {`
0x10474	mov r2, 0	mov r0, r0	16	`unsigned long long prev = 0;`
0x10478	mov r3, 0	mov r0, r0	16	`unsigned long long prev = 0;`
0x1047c	str r2, [fp, -0x24]	move r0, r0	16	`unsigned long long prev = 0;`
0x10480	str r3, [fp, -0x20]	move r0, r0	16	`unsigned long long prev = 0;`
0x10484	mov r2, 1	move r0, r0	17	`unsigned long long curr = 1;`
0x1048c	str r2, [fp, -0x1c]	move r0, r0	17	`unsigned long long curr = 1;`
0x10490	str r3, [fp, -0x18]	move r0, r0	17	`unsigned long long curr = 1;`
0x10494	mov r3, 2	move r0, r0	21	`for (int i =2; i <= n; i++){`
0x10498	str r3, [fp, -0x2c]	move r0, r0	21	`for (int i =2; i <= n; i++){`
0x104a4	ldm r1, {r0, r1}	move r0, r0	22	`next = prev + curr;`
0x104ac	ldm r3, {r2, r3}	move r0, r0	22	`next = prev + curr;`
0x104b0	adds r4, r0, r2	move r0, r0	22	`next = prev + curr;`
0x104b4	adc r5, r1, r3	move r0, r0	22	`next = prev + curr;`
0x104bc	str r5, [fp, -0x10]	move r0, r0	22	`next = prev + curr;`
0x104b8	str r4, [fp, -0x14]	move r0, r0	22	`next = prev + curr;`
0x104c8	str r2, [fp, -0x24]	move r0, r0	23	`prev = curr;`
0x104c4	ldm r3, {r2, r3}	move r0, r0	23	`prev = curr;`
0x104d4	ldm r3, {r2, r3}	move r0, r0	24	`curr = next;`
0x104d8	str r2, [fp, -0x24]	move r0, r0	24	`curr = next;`
0x104e0	ldr r3, [fp, -0x2c]	move r0, r0	21	`for (int i =2; i <= n; i++){`
0x104ec	ldr r2, [fp, -0x2c]	move r0, r0	21	`for (int i =2; i <= n; i++){`
0x104f4	cmp r2, r3	move r0, r0	21	`for (int i =2; i <= n; i++){`
0x104f8	ble 0x104a0	move r0, r0	21	`for (int i =2; i <= n; i++){`
0x10500	ldm r3, {r2, r3}	move r0, r0	28	`printf("Fibonacci of %d is: %llu", n, curr);`
0x1050c	bl 0x10304	move r0, r0	28	`printf("Fibonacci of %d is: %llu", n, curr);`
0x10538	mov r3, 0x14	move r0, r0	36	`int a = 20;`
0x10540	ldr r0, [fp, -0x20]	move r0, r0	38	`unsigned long long b = fib(a);`
0x10544	bl 0x10420	move r0, r0	38	`unsigned long long b = fib(a);`
0x1053c	str r3, [fp, -0x10]	move r0, r0	36	`int a = 20;`

```
#### Original Program vs Program 12 fib.o_0x10494 diassemebly
 ```
0|0x1048c 1c 20 0b e5   str r2, [fp, #-0x1c]|0x1048c 1c 20 0b e5   str r2, [fp, #-0x1c]|
1|0x10490 18 30 0b e5   str r3, [fp, #-0x18]|0x10490 18 30 0b e5   str r3, [fp, #-0x18]|
2|0x10494 02 30 a0 e3   mov r3, #2 |0x10494 00 00 a0 e1   mov r0, r0 |
3|0x10498 2c 30 0b e5   str r3, [fp, #-0x2c]|0x10498 2c 30 0b e5   str r3, [fp, #-0x2c]|
4|0x1049c 12 00 00 ea   b #0x104ec |0x1049c 12 00 00 ea   b #0x104ec |
 ```
```

Fig. 15: Disassembly of mutated file `fib.o_0x10494`, where the instruction in the golden binary at address 0x10494 is `mov r3, #2` while the instruction from the mutated binary at the same address is move r0 r0

```
❯ qemu-arm-static -L /usr/arm-linux-gnueabi ./mutated_bins/fib.o_0x10494
Fibonacci of 20 is: 17711
```

Fig. 16: Manually running a mutated Fibonnaci program.

```
target = "arm_32"
num_cpus = 24
num_bits = 1
upset_on_match = false
```

After updating the file, we can re-run the analysis and obtain a new set of results. Given the name of the configuration file shown above it named "bit_fib_profile.toml", it can be ran with the below command.

```
>>> pixi run cli run bit_fib_profile.toml
```

The equivalent `x-bit` command is:

```
>>> pixi run cli x-bit --program-file test_files/fib.c --program-input '' --expected-returncode
↪  0 --expected-stdout 6765  --no-list-expected --timeout 4 --out-dir fibonnaci_output --yes
↪  --target arm-32 --num-cpus 24 --num-bits 1 --no-upset-on-match
```

Conclusion. This completes this tutorial. We hope the information provided here is useful and readers can utilize our tool to examine other software programs. Besides this password check example and Fibonnaci example, we have investigated two machine learning C program examples, one for face recognition and one for digital recognition.